Cours 5

Réutiliser des programmes et des bibliothèques

1re année géomatique
2027-2028

Objectifs de la séance

Cours 3	→	**Cours 5**	→	Cours 6
permettre à une autre personne d'utiliser ce code		réutiliser des outils existants, et les réinstaller		partager ce code et travailler à plusieurs

Ajouter des fonctionnalités à un programme sans tout recoder : appeler un programme existant, utiliser une bibliothèque.
Décrire ces dépendances, pour les réinstaller sur un autre poste.

Vu aux cours précédents	Aujourd'hui
lancer un programme au terminal	`subprocess.run`, depuis un programme Python
`import math`, la bibliothèque standard	`import folium`, une bibliothèque à installer
`which python`, `(base)`	`PATH`, `sys.path`, un environnement
copier un script sur un autre poste	`environment.yml`, `pyproject.toml`

Réutiliser des programmes et des bibliothèques

Ajouter des fonctionnalités sans tout recoder, et réinstaller ensuite ces outils sur un autre poste.

Réutilisation d’outils existants

Un programme peut appeler des outils écrits et testés par d’autres, sans tout recoder.

Un programme existant

se lance par une commande : ogr2ogr, ffmpeg, git…

Une bibliothèque existante

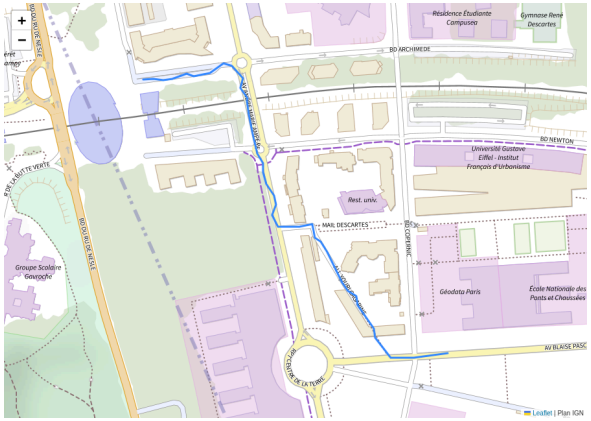
s'importe dans le code : folium, numpy, pillow…

	Tout recoder	Réutiliser un outil existant
Écriture du code	tout le calcul et ses cas particuliers	l’appel à l’outil
Correction des erreurs	par l’auteur du programme	par les auteurs de l’outil
Installation	Python	Python et l’outil, sur chaque poste qui lance le programme

Exemple : deux nouvelles fonctionnalités

`stats_trace.py`, le programme du cours 4, lit un fichier CSV. Une montre enregistre un fichier GPX. L'objectif est de lire ce fichier et d'afficher la trace sur une carte.

Fonctionnalité	Tout recoder	Outil existant
lire un fichier GPX et convertir ses positions en mètres	lire le format XML, calculer la projection	`ogr2ogr`, un programme de GDAL, qui gère 69 formats de fichiers
afficher la trace sur une carte	récupérer les tuiles de fond, gérer le zoom, dessiner la trace	`folium`, une bibliothèque de 9 495 lignes de Python



La carte obtenue affiche, sur le Plan IGN, une marche simulée sur un chemin réel, de la gare de Noisy-Champs à Geodata Paris.

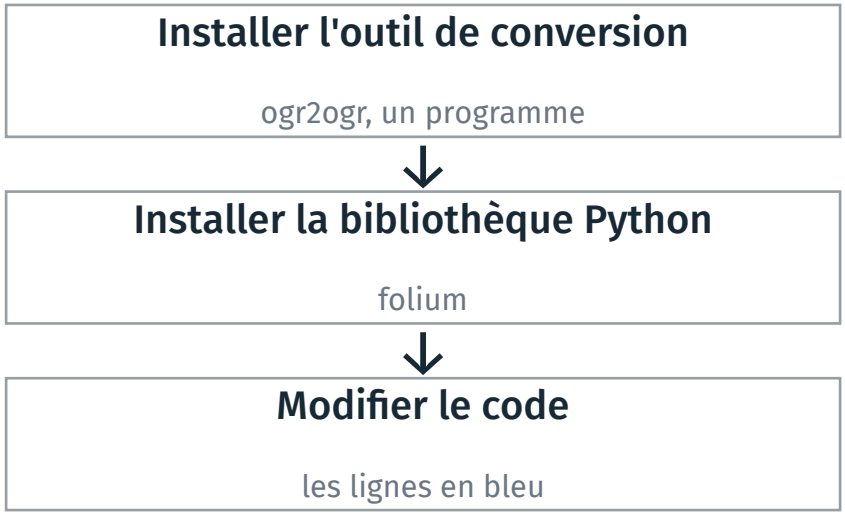
Ajout des fonctionnalités : code et installations

Le programme, en pseudo-code

```
importer la bibliothèque de carte

lire le chemin du fichier GPX
appeler le programme de conversion : GPX → CSV
lire le CSV et calculer le résumé (cours 4)
afficher le résumé
appeler la fonction d'affichage de la carte
```

Sur le poste de développement



En bleu, les lignes ajoutées au programme du cours 4.

Distribution du code : nouveau poste

Copié tel quel, le code ne fonctionne pas sur un nouveau poste, sauf si on y refait les mêmes installations. Il faut donc décrire ces installations.

Poste de développement `C:/Users/alice/cours5/`

`carte_trace.py, stats_trace.py, trace_2.gpx`

```
$ python carte_trace.py trace_2.gpx
…
vitesse moyenne : 1.29 m/s, soit 4.6 km/h
carte écrite : trace_2.html
```

Nouveau poste `C:/Users/eleve/projet/`

`carte_trace.py, stats_trace.py, trace_2.gpx`

```
$ python carte_trace.py trace_2.gpx
ModuleNotFoundError: No module named 'folium'
# une fois folium installé :
FileNotFoundError: [Errno 2] No such file or
directory: 'ogr2ogr'
```

Décrire dans la documentation

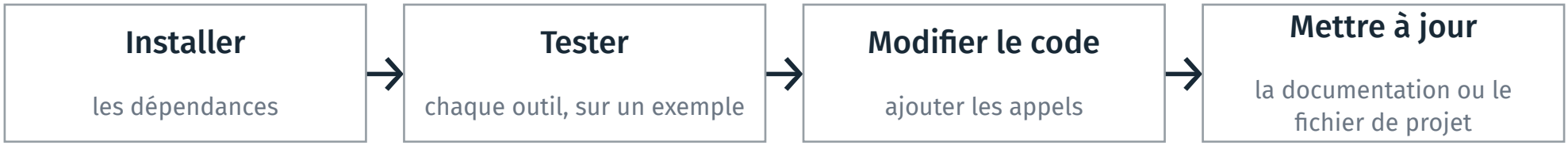
le README liste ce qu'il faut installer

Décrire dans un fichier de projet

un outil standard lit ce fichier, et installe les dépendances

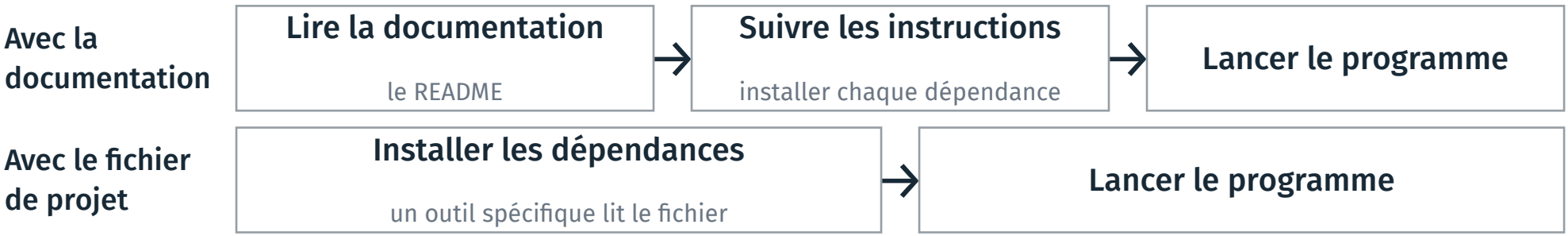
Distribution du code : étapes sur chaque poste

Poste de développement



↓ copie du dossier, ou dépôt git

Nouveau poste, autre utilisateur



Appeler un programme existant

Lancer un programme depuis Python, et la façon dont le système le trouve.

Appel d’un programme : étapes

Le programme, en pseudo-code

```
importer la bibliothèque de carte

lire le chemin du fichier GPX
appeler le programme de conversion : GPX → CSV
lire le CSV et calculer le résumé (cours 4)
afficher le résumé
appeler la fonction d'affichage de la carte
```

Étapes pour cette ligne



En bleu, la ligne traitée dans cette partie.

subprocess : lancement d'un programme

La fonction `subprocess.run` lance un programme en ligne de commande depuis un programme Python. Elle reçoit la commande sous forme de liste : le nom du programme, puis ses arguments, un élément par mot.

Dans le terminal	`git`	`log`	`--oneline`
Dans la liste Python	`"git"`	`"log"`	`"--oneline"`

Dans le terminal

Git Bash

```
$ git log --oneline
8d08215 Remplace la moyenne par les altitudes extrêmes
8195be4 Ajoute le calcul de la moyenne
```

Dans un programme Python

```
import subprocess
commande = ["git", "log", "--oneline"]
subprocess.run(commande, check=True)
```

`subprocess` est livré avec Python. Lancé dans le même dossier, ce programme affiche les mêmes lignes. `check=True` lève une erreur Python si la commande échoue.

subprocess : conversion d'un fichier GPX

`ogr2ogr` convertit les positions du GPX en Lambert-93, et écrit le CSV des TD 3 et 4.

Une position de trace_2.gpx

```
<trkpt lat="48.8434345" lon="2.5817636">
  <ele>94.7</ele><time>2026-09-07T07:40:00Z</time>
</trkpt>
```

Les mêmes valeurs dans trace_2.csv

```
X,Y,ele,time
669304.8,6860457.7,94.7,2026/09/07 07:40:00+00
```

Git Bash

```
$ ogr2ogr trace_2.csv trace_2.gpx track_points -f CSV -t_srs EPSG:2154 -lco GEOMETRY=AS_XY -select ele,time -xyRes 0.1
```

```
FICHIER_GPX = "trace_2.gpx"
FICHIER_CSV = "trace_2.csv"
options_ogr = ["-f", "CSV", "-t_srs", "EPSG:2154", "-lco", "GEOMETRY=AS_XY",
               "-select", "ele,time", "-xyRes", "0.1"]
commande = ["ogr2ogr", FICHIER_CSV, FICHIER_GPX, "track_points"] + options_ogr
subprocess.run(commande, check=True)
```

En bleu, les deux variables Python et leur emploi dans la commande. Dans le fichier GPX, `<trkpt>` est écrit sur une seule ligne.

subprocess : échec d'une commande

`subprocess.run` lève `FileNotFoundError` si le programme est introuvable. Il lève `CalledProcessError` si le programme échoue et que l'appel contient `check=True`.

Commande	**Avec** `check=True`	**Sans** `check=True`
programme introuvable : `["ogr2ogrr", "--version"]`	`FileNotFoundError`	`FileNotFoundError`
fichier absent : `["ogr2ogr", "-f", "CSV", "sortie.csv", "absent.gpx"]`	`CalledProcessError`	le programme Python continue, et `returncode` vaut 1

Git Bash

```
$ python appel.py
ERROR 1: Unable to open datasource `absent.gpx' with the following drivers.
…
subprocess.CalledProcessError: Command '['ogr2ogr', '-f', 'CSV', 'sortie.csv',
'absent.gpx']' returned non-zero exit status 1.
```

Sortie réelle, raccourcie. `ogr2ogr` écrit son message, puis Python écrit le sien.

subprocess : dossier courant et chemins

Un chemin relatif part du dossier courant, celui du terminal (cours 3). Écrit dans le code, le chemin du fichier GPX ne fonctionne donc que depuis le dossier de ce fichier.

Chemin écrit dans le code

```
FICHIER_GPX = "trace_2.gpx"
FICHIER_CSV = "trace_2.csv"
```

○○○ Git Bash

```
$ python travail/conversion.py
ERROR 1: Unable to open datasource
`trace_2.gpx' with the following drivers.
```

Chemin donné en argument

```
FICHIER_GPX = Path(options.gpx)
FICHIER_CSV = FICHIER_GPX.with_suffix(".csv")
```

○○○ Git Bash

```
$ python travail/carte_trace.py travail/trace_2.gpx
…
carte écrite : travail/trace_2.html
```

Les deux commandes sont lancées depuis le dossier parent de `travail`. `argparse` et `Path` : cours 3.

Installation d’un programme : globale ou locale

Installer un programme consiste à copier ses fichiers dans un dossier.

	Installation locale	Installation globale, dans un dossier par défaut	Installation globale, avec une variable mise à jour
Dossier	un dossier du projet ou du compte	un dossier du système, cherché par défaut	un autre dossier, ajouté à une variable d’environnement
Windows	`outils\gdal\bin\ogr2ogr.exe`	`C:\Windows\System32\cmd.exe`	`C:\Program Files\Git\cmd\git.exe`
Linux	`outils/gdal/bin/ogr2ogr`	`/usr/bin/ogr2ogr`	`/snap/bin/code`
Lancement	par son chemin	par son nom	par son nom

La variable d’environnement mise à jour s’appelle PATH : diapositive suivante.

Recherche d'un programme : variable PATH

Pour une commande sans `/`, Git Bash cherche dans chaque dossier de `PATH`, dans l'ordre, et lance le premier fichier de ce nom, avec ou sans `.exe`.

Programme dans un sous-dossier

```
projet/
├─ carte_trace.py
└─ outils/gdal/bin/
   └─ ogr2ogr.exe
```

Git Bash

```
$ outils/gdal/bin/ogr2ogr --version
GDAL 3.13.3 "Iowa City", released 2026/08/13
$ ogr2ogr --version
bash: ogr2ogr: command not found
```

La liste des dossiers

Git Bash

```
$ echo "$PATH" | tr ':' '\n'
/c/ProgramData/anaconda3
/c/ProgramData/anaconda3/Scripts
/usr/bin
/c/Python27
$ which -a python
/c/ProgramData/anaconda3/python
/c/Python27/python
```

À gauche, les commandes sont lancées depuis `projet/`. À droite, un poste où un Python 2.7 est aussi installé. Si aucun dossier ne contient la commande, Git Bash répond `command not found`, et `subprocess.run` lève `FileNotFoundError`.

Variable PATH : portée d'une modification

Une variable d'environnement associe un nom à une valeur, et cette valeur peut être une liste. Les dossiers de `PATH` sont séparés par `:` dans Git Bash et sous Linux, par `;` dans `cmd` et PowerShell.

Niveau	Modification	Portée	Durée
Système	Paramètres, « Modifier les variables d'environnement système »	tous les comptes du poste	jusqu'à la modification suivante
Compte	Paramètres, « Modifier les variables d'environnement pour votre compte »	ce compte	jusqu'à la modification suivante
Session	`export`, dans un terminal	ce terminal et les programmes qu'il lance	jusqu'à la fermeture du terminal

○○○ Git Bash

```
$ export PATH="$PWD/outils/gdal/bin:$PATH"
$ ogr2ogr --version
GDAL 3.13.3 "Iowa City", released 2026/08/13
```

`export` place le nouveau dossier devant la valeur actuelle de `PATH`, avec `:` entre les deux. Après une modification dans Paramètres, un terminal déjà ouvert garde l'ancienne valeur.

Bibliothèques et autres dépendances

La bibliothèque standard et les bibliothèques à installer, où elles se trouvent, et ce dont elles dépendent.

Bibliothèques : origines

Un programme appelle des fonctions déjà écrites, regroupées en **bibliothèques**.

Livrées avec Python

la bibliothèque standard : argparse, subprocess, pathlib…

Écrites pour le projet

vos propres fichiers, importés les uns par les autres

Publiées par d'autres

folium, numpy, pillow… à installer avant l'import

Exemple d'import de bibliothèque : folium

`folium` écrit une page web qui affiche une trace sur un fond de carte. Le programme importe la bibliothèque, puis appelle ses fonctions.

```
import folium
carte = folium.Map(tiles=PLAN_IGN, attr="Plan IGN", max_zoom=18)

trace = folium.GeoJson(str(FICHIER_GEOJSON))

trace.add_to(carte)
carte.fit_bounds(trace.get_bounds())
carte.save(FICHIER_CARTE)
```

la bibliothèque, à installer
la carte, sur le Plan IGN
la trace ; `folium` demande le chemin en chaîne
ajoute la trace à la carte
cadre la carte sur la trace
écrit la page `trace_2.html`

`FICHIER_GEOJSON` contient la trace, convertie par `ogr2ogr -f GeoJSON trace_2.geojson trace_2.gpx tracks`. `PLAN_IGN` contient l'adresse des tuiles du service WMTS de l'IGN. Documentation : python-visualization.github.io/folium.

Import : bibliothèque standard ou à installer

Une bibliothèque se déclare par `import`, en tête du programme, avant tout appel à ses fonctions. Si elle n'est pas disponible, l'import échoue et le programme s'arrête sur un `ModuleNotFoundError`.

Livrées avec Python

```
import math
print(math.sqrt(2))
```

```
1.4142135623730951
```

`math`, `csv`, `pathlib`, `json` : environ deux cents modules, installés en même temps que l'interpréteur.

Sorties réelles dans un environnement réduit à Python.

Publiées par d'autres

```
import folium
```

```
ModuleNotFoundError:
No module named 'folium'
```

`folium`, `numpy`, `pillow` : il faut les installer, et savoir lesquelles.

Installation d’une bibliothèque : pip

Un **paquet** (*package*) contient une bibliothèque, prête à installer. `pip` télécharge le paquet depuis le dépôt PyPI, et copie ses fichiers dans le dossier de la distribution Python.

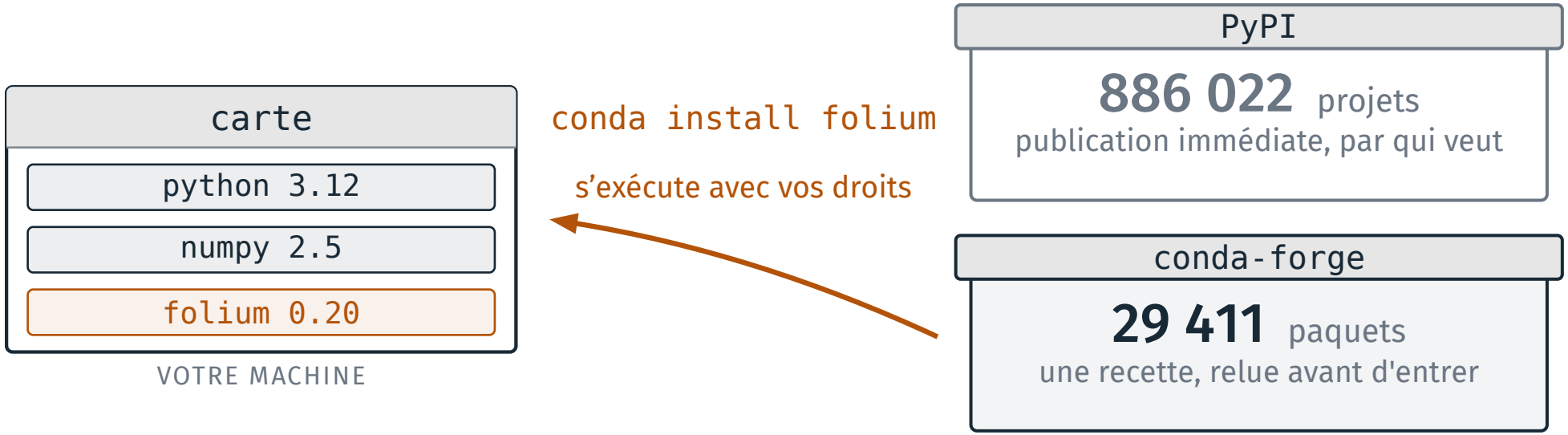
Git Bash

```
$ pip install folium      # avec conda : conda install -c conda-forge folium
Successfully installed branca-0.8.2 … folium-0.20.0 … xyzservices-2026.9.1
$ python -c "import folium; print(folium.__version__)"
0.20.0
$ pip show folium
Name: folium
Version: 0.20.0
Location: …/lib/python3.12/site-packages
Requires: branca, jinja2, numpy, requests, xyzservices
```

Sorties réelles, raccourcies. Le commentaire, après `#`, donne la même installation avec `conda`, présentée plus loin dans la partie.

Dépôts de paquets : PyPI et conda-forge

Les deux dépôts diffèrent par la façon d'y publier un paquet.



Les deux ne s'opposent pas : un paquet conda-forge est le plus souvent construit à partir de PyPI, quelques jours plus tard. Projets relevés le 8 septembre 2026 dans l'index de PyPI, recettes conda-forge le même jour.

Outils d’installation : repères

Plusieurs outils existent et l’écosystème évolue encore. Ici on utilise `conda` pour l’environnement et les dépendances, et `pip` pour installer le projet lui-même en local.



`pyenv`, né la même année que `conda`, n’est pas sur l’axe : il choisit la version de Python et n’installe aucun paquet. Dates de création des dépôts, relevées le 8 septembre 2026 par l’API de GitHub.

Dossier d’une distribution Python

Une distribution, comme Anaconda ou Miniforge, est installée dans un seul dossier. L’interpréteur, les programmes en ligne de commande et les bibliothèques y ont chacun leur sous-dossier.

Windows, avec Anaconda

```
anaconda3/
├─ python.exe                  l’interpréteur
├─ Scripts/                    pip.exe, conda.exe
├─ Library/bin/                ogr2ogr.exe
└─ Lib/                        la bibliothèque standard
   ├─ argparse.py, json/
   └─ site-packages/           les bibliothèques installées
      ├─ folium/
      └─ numpy/
```

Linux et macOS, avec Miniforge

```
miniforge3/
├─ bin/                        python, pip, ogr2ogr
└─ lib/python3.12/             la bibliothèque standard
   ├─ argparse.py, json/
   └─ site-packages/           les bibliothèques installées
      ├─ folium/
      └─ numpy/
```

Un environnement conda a la même organisation : partie 4. `sys.prefix` donne le dossier de la distribution, et `sys.executable` le chemin de l’interpréteur. Dans la salle : `C:\ProgramData\anaconda3`.

sys.path : recherche d'une bibliothèque

`import` cherche le module dans les dossiers de la liste `sys.path`, dans l'ordre, et charge le premier fichier de ce nom.

Ordre de recherche

1	le dossier du script
2	les dossiers de `PYTHONPATH`, une variable d'environnement souvent absente
3	la bibliothèque standard
4	les bibliothèques installées, dans `site-packages`

Un dossier ajouté par PYTHONPATH

Git Bash

```
$ export PYTHONPATH="$PWD/mes_modules"
$ python -c "import sys; print(*sys.path, sep='\n')"

…/projet/mes_modules
…/lib/python312.zip
…/lib/python3.12
…/lib/python3.12/lib-dynload
…/lib/python3.12/site-packages
$ python -c "import folium; print(folium.__file__)"
…/lib/python3.12/site-packages/folium/__init__.py
```

Sorties réelles sous Linux, chemins raccourcis. La ligne vide désigne le dossier courant. Quand Python lance un fichier, le dossier de ce fichier en prend la place. `PYTHONPATH` contient une liste de dossiers, séparés comme ceux de `PATH`.

Recherche d’une bibliothèque et d’un programme

Python cherche une bibliothèque dans `sys.path`. Le système cherche un programme dans `PATH`. Les deux erreurs vues sur l’autre poste viennent de ces deux recherches.

	`folium`	`ogr2ogr`
Nature	une bibliothèque Python	un programme en ligne de commande
Appel	`import folium`	`subprocess.run(["ogr2ogr", …])`
Recherche dans	`sys.path`	`PATH`
Ajout d’un dossier	`PYTHONPATH`	`PATH`
Installation dans	`Lib\site-packages\` , `lib/python3.12/site-packages/`	`Library\bin\` , `bin/`
Si le fichier est absent	`ModuleNotFoundError`	`FileNotFoundError`

Paquets sur PyPI : wheels et sources

Un projet peut publier sur PyPI ses sources et des archives prêtes à installer, les **wheels**. `pip` choisit le wheel adapté au poste, et compile les sources s'il n'en trouve aucun.

Paquet	Contenu	Fichiers sur PyPI	Effet de `pip install`
`folium`	code Python	1 wheel, pour tous les systèmes	copie les fichiers du wheel
`numpy`	code Python et code compilé	65 wheels, selon le système et la version de Python	copie les fichiers du wheel adapté
`ruff`	un programme compilé	17 wheels, selon le système	copie le programme dans `bin/`
`gdal`	le module `osgeo`, à compiler	les sources seules	compile le module, si `libgdal` et un compilateur sont déjà sur le poste

Fichiers relevés sur PyPI le 06/11/2026 : aucune des 98 versions de `gdal` n'a de wheel. Un wheel contient aussi les bibliothèques compilées qu'il appelle, par exemple 27 Mo dans `numpy.libs/`.

pip et conda : folium et GDAL

Un paquet PyPI ne peut déclarer que des paquets PyPI, et `libgdal` n'y figure pas. Un paquet conda peut contenir des bibliothèques compilées et des programmes.

	PyPI, avec `pip`	**conda-forge, avec `conda`**
`osgeo`, le module Python	`gdal`, sources à compiler	`gdal`
`libgdal`, la bibliothèque compilée	absente	`libgdal-core`
`ogr2ogr`, le programme	absent	`libgdal-core`
PROJ, GEOS et autres bibliothèques	impossibles à déclarer	30 paquets déclarés

Git Bash

```
$ pip install gdal
Could not find gdal-config. Make sure you have installed the GDAL native library and development headers.
$ conda install -c conda-forge gdal folium
$ ogr2ogr --version
GDAL 3.13.3 "Iowa City", released 2026/08/13
```

Sorties réelles, raccourcies, sous Linux. Avec `libgdal`, ses fichiers d'en-tête et un compilateur déjà sur le poste, `pip install gdal` réussit en 18 s, et n'installe pas `ogr2ogr`.

Installation de bibliothèques : les dépendances

Une bibliothèque déclare d'autres bibliothèques, qui en déclarent à leur tour. `pip` et `conda` lisent ces déclarations, et installent la liste complète.

Dépendances déclarées par folium

```
folium
├─ branca
│  └─ jinja2
│     └─ markupsafe
├─ jinja2                          déjà dans la liste
├─ numpy
├─ requests
│  ├─ certifi, charset-normalizer
│  └─ idna, urllib3
└─ xyzservices
```

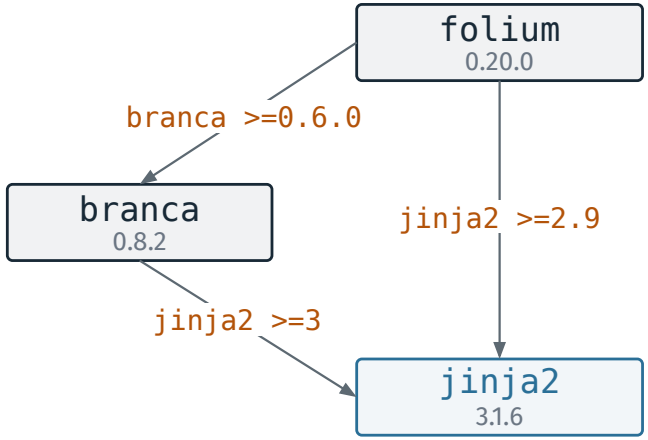
Installation de ces 11 paquets

À la main	copier `folium` dans `site-packages`, puis ses dépendances, puis les dépendances de ces dépendances
Avec un outil	taper `pip install folium` : `pip` lit les déclarations et copie les 11 paquets

Chaque dépendance ajoute des paquets à installer et des versions à suivre. Il faut donc trouver un compromis entre tout recoder et installer une dépendance pour chaque besoin.

Dépendances : résolution des versions

`site-packages` contient une seule version de chaque paquet. `pip` et `conda` choisissent ces versions à partir de toutes les exigences déclarées : cette étape s'appelle la **résolution**.



une seule version installée, qui satisfait les deux exigences

Git Bash

```
$ pip install "branca==0.8.2" "jinja2<3"
…
ERROR: Cannot install branca==0.8.2 and jinja2<3
because these package versions have conflicting
dependencies.
The conflict is caused by:
    The user requested jinja2<3
    branca 0.8.2 depends on jinja2>=3
…
ERROR: ResolutionImpossible
```

À droite, `jinja2<3` contredit l'exigence de `branca 0.8.2`, et `pip` s'arrête sans rien installer. Sorties réelles, raccourcies.

Le numéro de version d’une dépendance

D’une version à l’autre, une bibliothèque gagne des fonctions et en retire d’autres. Le même code ne passe donc pas partout.

Le code	avec `numpy` 1.26.4	avec `numpy` 2.5.2
`np.trapezoid([0, 1, 2])`	`AttributeError: module 'numpy' has no attribute 'trapezoid'`	`2.0`
`np.NaN`	`nan`	`AttributeError: np.NaN was removed in the NumPy 2.0 release`

La valeur n’a pas disparu. En 1.26, `np.nan`, `np.NaN` et `np.NAN` désignent la même chose ; la version 2.0 n’en garde qu’une.

Sorties réelles, relevées le 10 septembre 2026 dans deux environnements. D’où deux exigences opposées : `numpy>=2` pour la première ligne, `numpy<2` pour la seconde tant que le code n’est pas repris.

TRAVAUX DIRIGÉS

TD 5a L'emplacement de votre distribution Python ≈ 10′

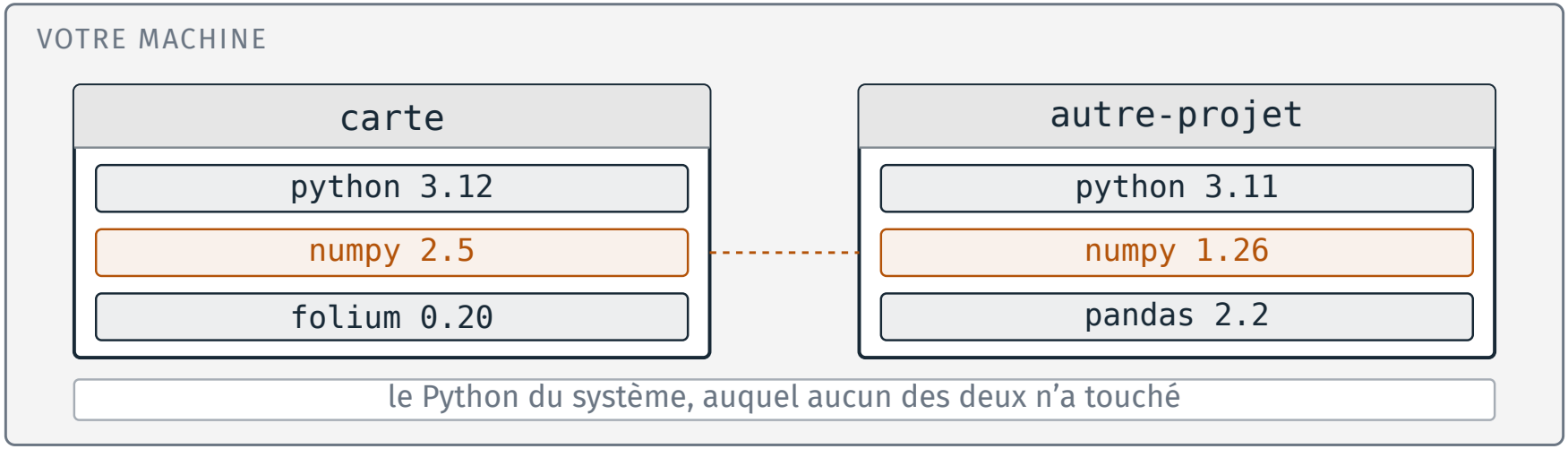
`cours5/5a_distribution/`

Environnement et fichier de projet

Un dossier par projet, et les fichiers qui permettent de le recréer sur un autre poste.

Environnement : un moyen de gérer les conflits de dépendances

Un projet demande `numpy 1.26`, l'autre `numpy 2.5`, et `site-packages` ne contient qu'une version de chaque paquet. Chaque projet reçoit donc son propre dossier, un **environnement**.



la même bibliothèque, en deux versions, sans qu'elles se croisent

Chaque environnement contient son Python et ses paquets. Le Python du système ne change pas.

Environnement : création et dossier

`conda create` crée un dossier, et y installe Python et les paquets demandés. `conda activate` sélectionne ensuite cet environnement dans le terminal.

Commandes

Git Bash

```
$ conda create -n carte -c conda-forge \
      python=3.12 gdal folium
…
$ conda activate carte
(carte) $ python carte_trace.py trace_2.gpx
…
carte écrite : trace_2.html
```

Dossier de l'environnement, sous Windows

```
…\.conda\envs\carte\
├─ python.exe              l'interpréteur
├─ Scripts/                pip.exe
├─ Library/bin/            ogr2ogr.exe
└─ Lib/site-packages/      folium/, osgeo/
```

Ce dossier a la même organisation que le dossier d'une distribution. Dans la salle, le compte élève ne peut pas modifier `anaconda3`, et `conda` écrit donc dans `.conda\envs\` . Sous Linux et macOS, le dossier est `~/miniforge3/envs/carte/`.

Activation d’un environnement : effet sur PATH

Activer un environnement n’installe rien et ne déplace aucun fichier. La modification de `PATH` vaut pour ce terminal, comme un `export`.

Avant `conda activate carte` : le `python.exe` de `anaconda3` est lancé

`/c/ProgramData/anaconda3`	`python.exe`
`/usr/bin`	`ls`, `cp`, `bash`…
`/c/Windows/system32`	`cmd.exe`…

Après `conda activate carte` : les dossiers de l’environnement passent en tête

`/c/Users/eleve/.conda/envs/carte`	`python.exe`
`…/envs/carte/Library/bin`	`ogr2ogr.exe`
`/c/ProgramData/anaconda3`	`python.exe`
`/usr/bin`	`ls`, `cp`, `bash`…
`/c/Windows/system32`	`cmd.exe`…

À gauche, aucun dossier ne contient `ogr2ogr.exe`. Sous macOS et Linux, `…/envs/carte/bin` passe devant `/usr/bin`.

Activation : recherche des dépendances

Après l'activation, `PATH` désigne le `python` et le `ogr2ogr` de l'environnement. Cet interpréteur calcule `sys.path` à partir de son propre dossier.

Avant `conda activate carte`

Git Bash

```
$ which python
…/miniforge3/bin/python
$ ogr2ogr --version
bash: ogr2ogr: command not found
$ python -c "import sys; print(sys.path[-1])"
…/miniforge3/lib/python3.10/site-packages
$ python -c "import folium; print(folium.__version__)"
ModuleNotFoundError: No module named 'folium'
```

Après `conda activate carte`

Git Bash

```
(carte) $ which python
…/envs/carte/bin/python
(carte) $ ogr2ogr --version
GDAL 3.13.3 "Iowa City", released 2026/08/13
(carte) $ python -c "import sys; print(sys.path[-1])"
…/envs/carte/lib/python3.12/site-packages
(carte) $ python -c "import folium; print(folium.__version__)"
0.20.0
```

Sorties réelles sous Linux, chemins raccourcis. `sys.path[-1]` désigne le dernier dossier de la liste. À gauche, les deux recherches échouent, comme sur le nouveau poste de la partie 1.

Environnement : README ou environment.yml

Les commandes tapées au terminal ne sont enregistrées nulle part dans le projet. Pour refaire l'environnement ailleurs, il faut les décrire dans le `README`, ou dans un fichier lu par `conda`.

Dans la documentation

à la racine du projet : `README.md`

```
## Installation

conda create -n carte -c conda-forge \
    python=3.12 gdal folium
conda activate carte
```

L'utilisateur lit le fichier, puis tape ces commandes.

Dans un fichier lu par conda

à la racine du projet : `environment.yml`

```yaml
name: carte
channels:
  - conda-forge
dependencies:
  - python=3.12
  - gdal
  - folium
```

`conda env create -f environment.yml` lit le fichier, et installe les paquets.

Les deux fichiers accompagnent le code, dans la copie du dossier ou dans le dépôt git.

Projet : déclaration des informations & métadonnées

Un projet Python se décrit dans un fichier, `pyproject.toml`. `pip install .` lit ce fichier dans le dossier courant, puis installe le projet et ses dépendances.

`pyproject.toml`, à la racine du projet

```toml
[project]
name = "carte-trace"
version = "0.1"
requires-python = ">=3.10"
dependencies = ["folium>=0.20"]

[tool.setuptools]
py-modules = ["carte_trace", "stats_trace"]
```

Rôle de chaque ligne

Clé	Rôle
`name`, `version`	le nom et la version du projet
`dependencies`	les bibliothèques à installer
`requires-python`	les versions de Python acceptées
`py-modules`	les fichiers du projet à installer

○○○ Git Bash

```
$ pip install .
Successfully installed … carte-trace-0.1 … folium-0.20.0 … xyzservices-2026.9.1
```

Fichiers de projet : environment.yml et pyproject.toml

Le projet GDAL ne publie pas `ogr2ogr` sur PyPI, et `carte_trace.py` a donc besoin de `environment.yml`. Un projet dont toutes les dépendances sont sur PyPI peut se limiter à `pyproject.toml`.

	`environment.yml`	`pyproject.toml`
Lu par	`conda`	`pip`
Contenu	Python, bibliothèques et programmes	paquets de PyPI, nom et version du projet
`folium`	déclaré	déclaré
GDAL et `ogr2ogr`	déclarés	absents
Commande	`conda env create -f environment.yml`	`pip install .`

Distribution du code : commandes sur un nouveau poste

Sur le nouveau poste, `conda` recrée l'environnement à partir de `environment.yml`. Le programme s'exécute ensuite comme sur le poste de développement.

Sans environnement `C:/Users/eleve/projet/carte_trace.py, stats_trace.py, trace_2.gpx`

```
$ python carte_trace.py trace_2.gpx
ModuleNotFoundError: No module named 'folium'
# une fois folium installé :
FileNotFoundError: [Errno 2] No such file or
directory: 'ogr2ogr'
```

Avec environment.yml `C:/Users/eleve/projet/carte_trace.py, stats_trace.py, trace_2.gpx, environment.yml`

```
$ conda env create -f environment.yml
…
$ conda activate carte
(carte) $ python carte_trace.py trace_2.gpx
distance parcourue : 632.3 m
vitesse moyenne : 1.29 m/s, soit 4.6 km/h
carte écrite : trace_2.html
```

Le dossier du projet arrive par une copie, ou par `git clone`. Sorties réelles sous Linux, raccourcies.

Commandes conda : résumé

Commande	Effet
`conda create -n carte python=3.12 gdal`	crée le dossier de l'environnement `carte`, et y installe ces paquets
`conda env create -f environment.yml`	crée l'environnement que décrit le fichier
`conda activate carte`	sélectionne `carte` dans ce terminal, en plaçant son dossier en tête de `PATH`
`conda install ipykernel`	installe un paquet dans l'environnement actif
`conda list`	liste les paquets de l'environnement actif, avec leur version
`conda deactivate`	revient à l'environnement précédent
`conda env list`	liste les environnements ; * marque l'environnement actif
`conda env remove -n carte`	supprime le dossier de l'environnement

`-n carte` désigne un environnement sans l'activer, comme dans `conda install -n carte ipykernel`. `-c conda-forge` désigne le dépôt des paquets. Dans un environnement actif, `pip install` écrit aussi dans son dossier.

TRAVAUX DIRIGÉS

TD 5b Un environnement pour le programme ≈ 20′

`cours5/5b_environnement/`

TD 5c Décrire l’installation pour un autre poste ≈ 20′

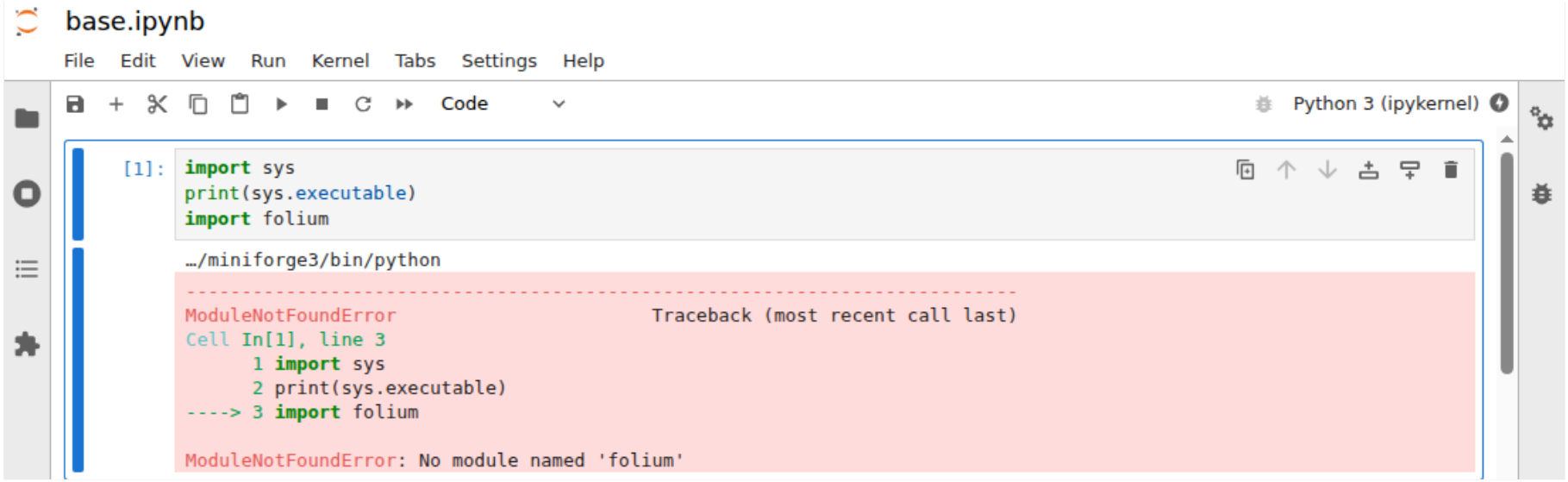
`cours5/5c_description/`

JupyterLab et le noyau, partie facultative

Le client, le serveur, et le Python qui exécute un notebook.

Notebook : bibliothèque introuvable

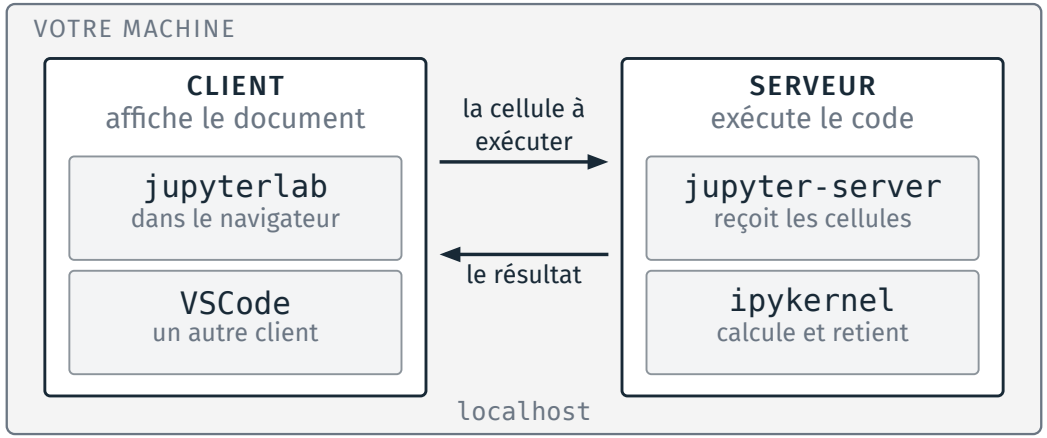
Dans le terminal, `carte_trace.py` fonctionne après `conda activate carte`. Dans un notebook ouvert par JupyterLab, `import folium` lève `ModuleNotFoundError`.



`sys.executable` affiche le chemin de l'interpréteur : cette cellule s'exécute avec le Python de `base`. Capture de JupyterLab 4.6 sous Linux, chemin raccourci.

Client et serveur : JupyterLab

Un client envoie une requête, et un serveur répond. Dans JupyterLab, les deux programmes tournent sur le même poste, et le serveur lance un **noyau**, un interpréteur Python chargé d'exécuter les cellules.

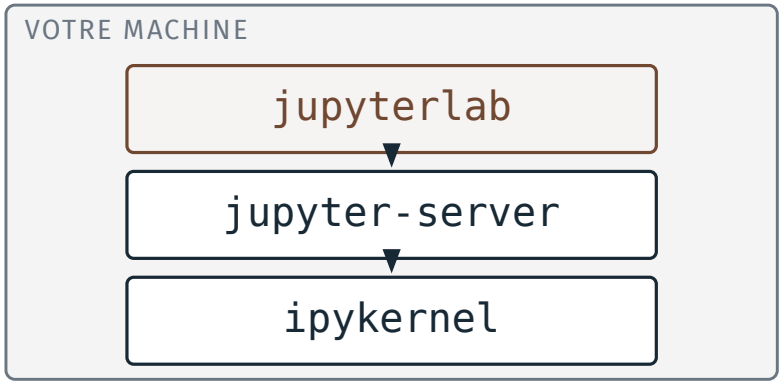


`jupyter lab`, lancé dans un terminal, démarre le serveur. Le navigateur est le client : il se connecte à l'adresse `localhost:8888`, qui désigne le poste lui-même.

Les clients d’un notebook

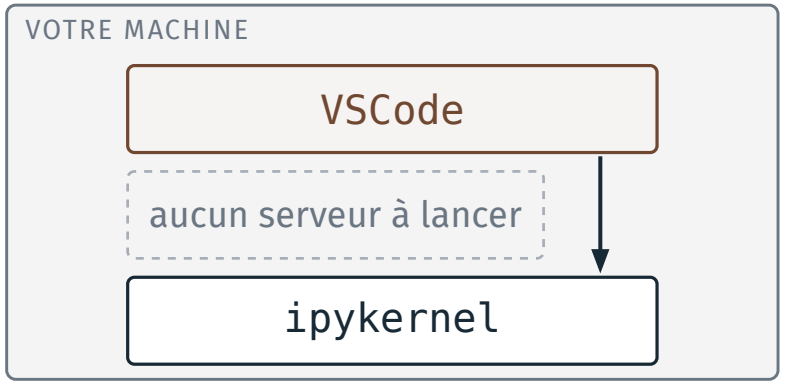
Le même fichier `.ipynb` s’ouvre dans JupyterLab ou dans VS Code. Les deux clients ont besoin d’un noyau.

DANS LE NAVIGATEUR



jupyter lab, puis une adresse localhost

DANS L'ÉDITEUR DE CODE



l'éditeur démarre le noyau lui-même

L’éditeur de code n’a pas besoin de `jupyterlab` : il démarre `ipykernel` lui-même. `ipykernel` est le noyau Python ; il en existe pour d’autres langages, et le nom *Jupyter* vient de Julia, Python et R.

Noyau d’un environnement : déclaration

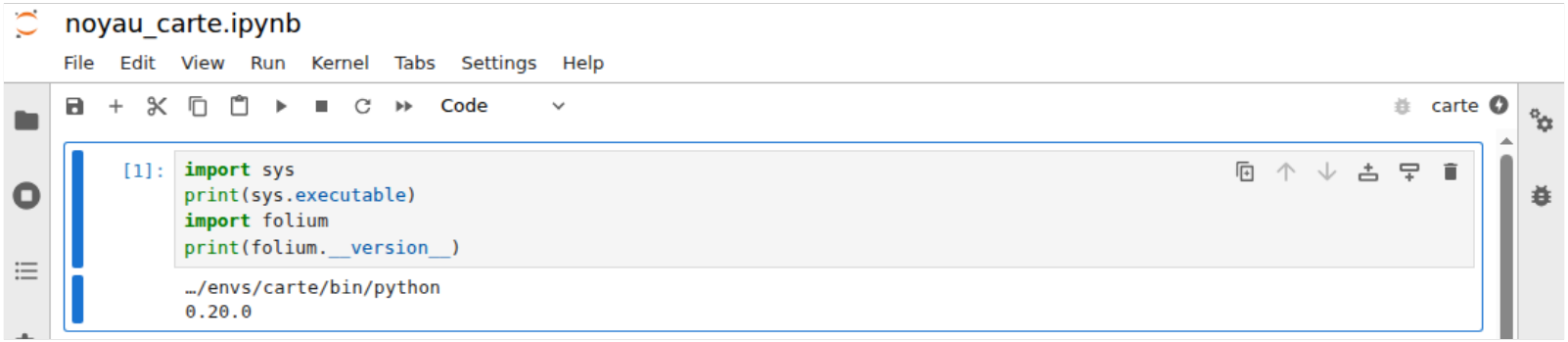
`python -m ipykernel install` écrit un fichier `kernel.json`, qui donne le chemin de l’interpréteur de `carte`. JupyterLab propose alors ce noyau, et `import folium` réussit.

Git Bash

```
$ conda activate carte
(carte) $ conda install -c conda-forge ipykernel
(carte) $ python -m ipykernel install --user --name carte
Installed kernelspec carte in …/jupyter/kernels/carte
```

kernel.json

```
"argv": [
  "…/envs/carte/bin/python", …
```



Le nom du noyau, `carte`, apparaît en haut à droite du notebook. Capture de JupyterLab 4.6 sous Linux, chemin raccourci.

Noyau : recherche des dépendances

Le noyau trouve ses bibliothèques à partir de son interpréteur. Il reçoit `PATH` du serveur, donc du terminal où `jupyter lab` a été lancé.

Cellule

```
import shutil
import sys
print(sys.executable)
print(shutil.which("ogr2ogr"))
import folium
print("folium", folium.__version__)
```

JupyterLab de `base`, noyau `carte`

```
…/envs/carte/bin/python
None
folium 0.20.0
```

JupyterLab de `carte`, après activation

```
…/envs/carte/bin/python
…/envs/carte/bin/ogr2ogr
folium 0.20.0
```

Avec `None`, `subprocess.run(["ogr2ogr", …])` lève `FileNotFoundError`. Pour lancer `jupyter lab` après `conda activate carte`, il faut installer `jupyterlab` dans `carte`, car l'activation retire `base` de `PATH`. Sorties réelles sous Linux, chemins raccourcis.

TRAVAUX DIRIGÉS

TD 5d Le noyau de l'environnement — facultatif ≈ 15′

`cours5/5d_noyau/`

À retenir

Récapituler l’appel d’un programme, l’installation d’une bibliothèque et l’environnement d’un projet.

Aide-mémoire : programmes et PATH

Appel depuis Python : `subprocess`	
`import subprocess`	importe le module, livré avec Python
`subprocess.run(commande)`	lance le programme ; la commande est une liste, un élément par mot
`check=True`	lève `CalledProcessError` si le programme échoue
`FileNotFoundError`	levée si le programme est introuvable
Recherche d'un programme : `PATH`	
`PATH`	liste les dossiers où chercher une commande, dans l'ordre
`which ogr2ogr`	affiche le chemin du programme trouvé
`echo "$PATH"`	affiche la liste ; `:` sépare les dossiers dans Git Bash
`export PATH="…:$PATH"`	place un dossier en tête, pour ce terminal et les programmes qu'il lance

Une commande qui contient un `/` est lancée sans passer par PATH. Un chemin relatif part du dossier du terminal.

Aide-mémoire : bibliothèques

Installation	
`pip install folium`	télécharge le paquet depuis PyPI, et l'installe avec ses dépendances
`pip show folium`	affiche la version, le dossier et les dépendances
PyPI, conda-forge	les dépôts de `pip` et de `conda`
résolution	le choix d'une version de chaque paquet, compatible avec toutes les exigences
Recherche par `import`	
`import folium`	charge la bibliothèque ; `ModuleNotFoundError` si elle manque
bibliothèque standard	livrée avec Python : `pathlib`, `subprocess`, `argparse`
`sys.path`	liste les dossiers où `import` cherche, dans l'ordre
`site-packages`	contient les bibliothèques installées

`pip` n'installe ni `ogr2ogr` ni la bibliothèque compilée de GDAL, absents de PyPI. `conda` les installe depuis conda-forge.

Aide-mémoire : environnement et projet

Environnement : `conda`	
environnement	le dossier d'un projet, avec son Python et ses paquets
`conda activate carte`	place son dossier en tête de `PATH`, pour ce terminal
`conda env create -f environment.yml`	recrée l'environnement que décrit le fichier
`which python`, `sys.executable`	donnent l'interpréteur lancé
Fichiers de projet et noyau	
`environment.yml`	Python, bibliothèques et programmes, lu par `conda`
`pyproject.toml`	nom, version et dépendances PyPI du projet, lu par `pip`
`pip install .`	installe le projet du dossier courant et ses dépendances
`python -m ipykernel install --user --name carte`	déclare le noyau de `carte`, l'interpréteur qui exécute les cellules d'un notebook

Les autres commandes de `conda` sont sur « Commandes conda : résumé », partie 4.

Avant la séance 6 : un compte GitHub

Créer son compte chez soi, et repérer sur la page d’un dépôt l’adresse à donner à git.

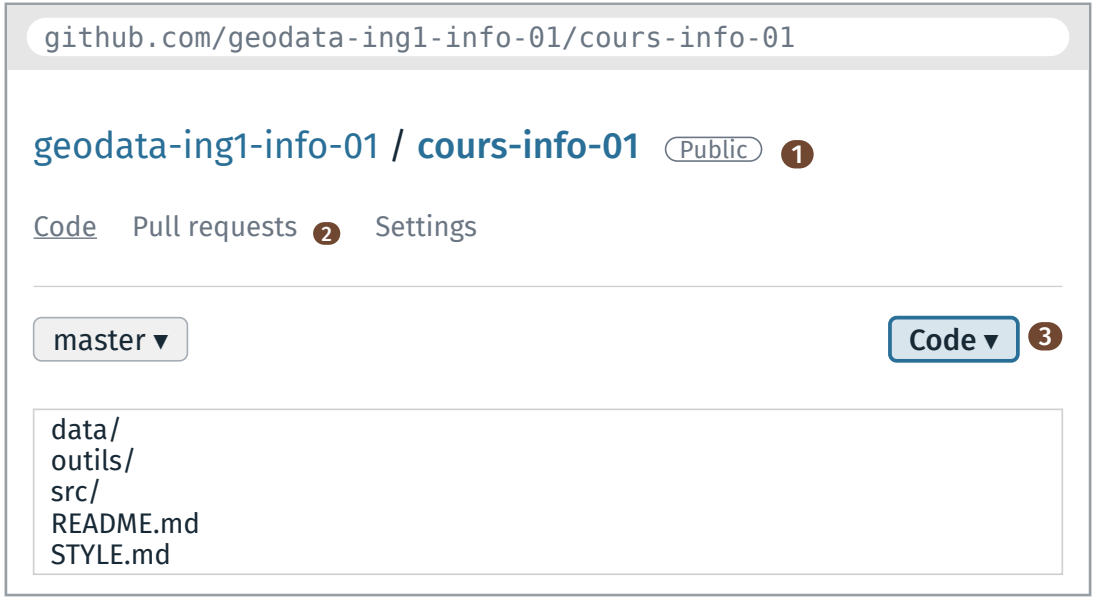
Création d’un compte GitHub

> **Objectif :** créer un compte GitHub avant la séance 6, depuis un autre réseau que celui de l’école, et y activer le deuxième facteur.

1. Se connecter à un autre réseau que celui de l’école : la box de chez soi, ou le partage de connexion de son téléphone. Derrière le proxy de l’école, tous les postes ont la même adresse IP, et GitHub refuse les inscriptions trop nombreuses depuis une même adresse.
2. Aller sur `github.com/signup`, puis saisir son adresse e-mail, un mot de passe et un nom de compte. Le nom de compte figure dans l’adresse de tous ses dépôts : choisir un nom sobre, par exemple `prenom-nom`.
3. Saisir le code reçu par e-mail.
4. Activer le deuxième facteur avec une application à codes, puis garder les codes de secours. Menu du compte, Settings, Password and authentication.
5. Vérifier que la page `github.com/<compte>` affiche son profil.

Page d’un dépôt GitHub

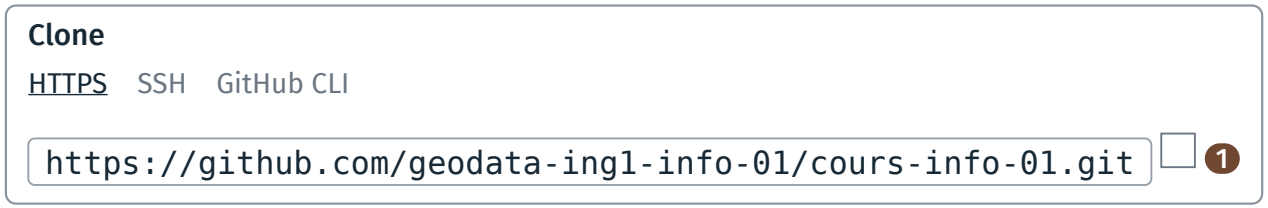
Un dépôt GitHub appartient à un compte. Sa page montre les fichiers de la dernière version, et donne l’adresse du dépôt pour `git clone`.



1. le compte, puis le nom du dépôt, repris dans l’adresse de la page
2. les pull requests : les modifications proposées, à fusionner (séance 7)
3. le bouton Code, qui donne l’adresse du dépôt

Adresse HTTPS d'un dépôt

Le bouton Code, onglet HTTPS, donne l'adresse du dépôt, en `https://github.com/<compte>/<dépôt>.git`. `git clone` copie le dépôt à cette adresse.



Git Bash

```
$ git clone https://github.com/geodata-ing1-info-01/cours-info-01.git
Clonage dans 'cours-info-01'...
```

1 copier l'adresse, puis la coller après `git clone`. L'onglet SSH donne une adresse en `git@github.com:…`, qui passe par le port 22, fermé par le pare-feu de l'école : en salle, prendre l'adresse HTTPS.